

1. What are Exceptions?

Definition (Interview Answer):

Exceptions are runtime events that disrupt the normal flow of a program. They represent errors such as invalid input, resource failure, or business rule violations.

Types of Exceptions

Type	Description	Example
Checked	Must be handled at compile time	IOException
Unchecked	Occur at runtime	NullPointerException
Custom	User-defined	UserNotFoundException

2. Why Exception Handling in Spring Boot?

Answer:

- To avoid application crash
- To provide meaningful API responses
- To maintain clean and maintainable code
- To centralize error handling logic

3. What is @ExceptionHandler?

Answer:

@ExceptionHandler is used to handle specific exceptions inside a controller or globally

1. Spring looks for matching @ExceptionHandler..and map with Exception

Example:

```
@ExceptionHandler(IllegalArgumentException.class)
```

```
public ResponseEntity<String> handleException(IllegalArgumentException ex) {
    return new ResponseEntity<>(ex.getMessage(), HttpStatus.BAD_REQUEST);
}
```

Why use it?

- Handles **specific exception types**
- Gives **custom response instead of default error**
- Improves API usability

4. What is @RestControllerAdvice?

Answer:

@RestControllerAdvice is used to **create global exception handling across all controllers and returns responses in JSON format.**

Example:

@RestControllerAdvice → global Exception Handling ke liye use hota hai and return Response in json

```
public class GlobalExceptionHandler {

    @ExceptionHandler(Exception.class) → inside global or matching used for mapping
    public ResponseEntity<String> handleAll(Exception ex) {
        return new ResponseEntity<>(ex.getMessage(),
        HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Why use it?

- Centralizes exception handling
- Avoids duplicate code in controllers
- Ensures consistent API responses

5. Difference: @ControllerAdvice vs @RestControllerAdvice

Feature	@ControllerAdvice	@RestControllerAdvice
Response type	View (HTML)	JSON
Use case	MVC apps	REST APIs
Includes	@ControllerAdvice	@ControllerAdvice + @ResponseBody

6. What is a Custom Exception?

Answer:

A custom exception is a user-defined class used to represent **business logic errors**.

Why? because sometimes we need ...extra exception handling like (Not created user etc)

So we need custom Exception

Example: Alag file me: jo parent call krte the t0 Global file

```
public class UserNotFoundException extends RuntimeException {
    public UserNotFoundException(String message) {
        super(message);
    }
}
```

Why extend **RuntimeException**?

- No need for **try-catch**
- Cleaner code
- Preferred in Spring Boot APIs

7. How Exceptions Flow in Spring Boot

Flow:

2. Exception occurs in Controller/Service
3. Spring looks for matching **@ExceptionHandler**
4. Global handler (**@RestControllerAdvice**) catches it
5. Custom response is returned

8. Why use `Map<String, Object>` in response?

Answer:

It is used to create a **dynamic JSON response**.

Example:

```
Map<String, Object> error = new HashMap<>();
error.put("message", ex.getMessage());
error.put("status", 400);
```

Why Map?

- Flexible structure
- Easy to modify
- Quick implementation

Interview Tip:

👉 Better approach:

Use a **custom class instead of Map**

```
class ErrorResponse {
    String message;
    int status;
    LocalDateTime time;
}
```

- ✓ More readable
- ✓ Industry standard

9. Handling Multiple Exceptions

```
@ExceptionHandler({IllegalArgumentException.class, NullPointerException.class})
```

👉 One method handles multiple exceptions

10. Common Exceptions to Handle

Exception	Status
IllegalArgumentException	400
NullPointerException	400
UserNotFoundException	404
HttpRequestMethodNotSupportedException	405

11. Best Practices (VERY IMPORTANT)

- ✓ Use `@RestControllerAdvice` for global handling
- ✓ Use **custom exceptions for business logic**
- ✓ Return proper HTTP status codes
- ✓ Avoid exposing internal errors
- ✓ Use structured response (DTO instead of Map)
- ✓ Log errors using `Logger`

12. Professional Answer

“Spring Boot exception handling is implemented using `@ExceptionHandler` and `@RestControllerAdvice` to provide centralized, consistent, and meaningful error responses while improving maintainability and user experience.”

If interviewer asks follow-up

Be ready for:

- Difference between checked & unchecked

- Why RuntimeException
 - How to handle validation errors (@Valid)
 - How to design error response DTO
-

1. What is Logging?

Interview Answer:

Logging is the process of recording application events, errors, and execution details to help in debugging, monitoring, and maintaining applications.

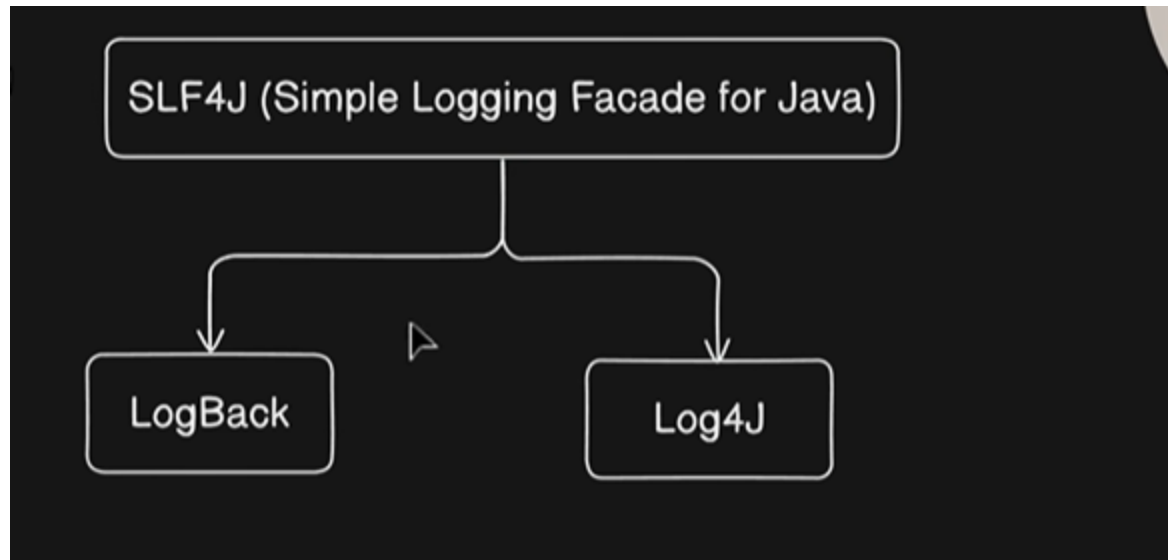
2. Why do we use Logging?

👉 Use Cases:

- ☒ Debugging issues during development
 - ☒ Monitoring application in production
 - ☒ Tracking errors and failures
 - ☒ Understanding application flow
 - ☒ Auditing and security tracking
-

3. What logging framework does Spring Boot use?

Answer:



Spring Boot uses **Logback** by default through **SLF4J(Interface)**.

Explanation

- **SLF4J** → Interface (abstraction)
- **Logback** → Actual implementation

This allows switching logging frameworks easily.

4. Logging Levels

Level	Use
TRACE	Very detailed logs
DEBUG	Debugging info
INFO	Normal application flow (default)
WARN	Something suspicious
ERROR	Serious issue

Interview Line

“INFO is default level, DEBUG and TRACE provide more detailed insights.”

5. How to Enable Debug Logs?

👉 In `application.properties`:

```
logging.level.root=DEBUG
```

6. How to Create Logger in Spring Boot?

```
import org.slf4j.Logger;
```

```
import org.slf4j.LoggerFactory;
```

```
private static final Logger logger = LoggerFactory.getLogger(UserServices.class);
```

7. How to Use Logger?

```
logger.info("User created successfully");  
logger.debug("User details: {}", user);  
logger.warn("User already exists");  
logger.error("Error occurred", exception);
```

Best Practice

👉 Always use `{ }` instead of string concatenation:

```
logger.info("User id: {}", id); // ✅
```

8. Logging with Exception Handling (VERY IMPORTANT)

👉 Combine logging with global exception handler:

...

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    private static final Logger logger =
        LoggerFactory.getLogger(GlobalExceptionHandler.class);

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleException(Exception ex){

        logger.error("Exception occurred", ex); // 🔥 important

        Map<String, Object> error = new HashMap<>();
        error.put("message", ex.getMessage());

        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Why use logging here?

Real-world scenario:

- Error happens for 10,000 users
- You cannot reproduce locally
- Logs help you debug production issues

9. What happens in your console logs?

Example:

INFO → Application started
DEBUG → Bean creation details
ERROR → Exception stack trace

👉 Your screenshot logs = Spring Boot internal working (DispatcherServlet, mappings, etc.)

10. Key Interview Questions

Q1: Difference between System.out.println and Logger?

👉 Answer:

System.out	Logger
Not configurable	Configurable levels
No severity levels	Supports levels
Not used in production	Used in production

Q2: Why use SLF4J?

👉 Answer:

- Provides abstraction
- Allows switching logging frameworks
- Industry standard

Q3: What is default logging level?

👉 INFO

Q4: Where logs are stored?

👉 By default:

- Console
- Can be configured to file using properties

11. Save Logs to File

logging.file.name=app.log

Final Professional Answer

“Spring Boot uses SLF4J with Logback for logging.” Logging helps in debugging, monitoring, and tracking application behavior. Different log levels like DEBUG, INFO, WARN, and ERROR are used based on severity, and logs are especially useful in production for diagnosing issues when exceptions occur.”